Kubernetes YAML Files and Helm Charts Tutorials

Summary

🧭 Overview

This session was a hands-on Kubernetes training focused on deploying applications using YAML manifest files and introducing Helm charts as a modern deployment tool. The training was led by JJ Tech Inc and Professor Sunny, who walked participants through creating a Kubernetes cluster on Azure, deploying applications via YAML files, and then transitioning to Helm charts for more efficient, reusable deployments.

The session built upon previous week's work with Kubernetes, reinforcing concepts like deployments, services, pods, and replicas. The primary learning goal was to help participants understand how to move from imperative CLI commands and manual YAML files to declarative, templated Helm charts that enable consistent deployments across multiple environments (development, QA, staging, production).

By the end of the session, participants were expected to understand Helm's package management capabilities, create their own Helm charts from scratch, modify values for different environments, manage versioning and rollbacks, and prepare for working with third-party Helm charts from repositories like Artifact Hub.

🧩 Key Topics and Concepts

Kubernetes Cluster Architecture Review

- **Control plane and worker nodes:** The session reviewed how the control plane manages the cluster while worker nodes run the actual application pods. Participants practiced connecting to an Azure Kubernetes Service (AKS) cluster using Azure CLI commands.

- **Namespaces:** Logical partitions within a Kubernetes cluster that help organize resources. The default namespace was used throughout the demo, but participants learned they could create custom namespaces for different teams or environments.

YAML Manifest Files

- **Deployment YAML structure:** Includes API version, kind (Deployment), metadata (name and labels), spec (replicas, selector, template). The deployment defines how many pod replicas should run and what image to use.
- **Service YAML structure:** Exposes deployments to network traffic. Key fields include type (ClusterIP, NodePort, LoadBalancer), ports, and selectors that match deployment labels.
- **Label matching:** Services find pods through label selectors. If a deployment has label "app: nginx", the service must have a selector "app: nginx" to route traffic correctly.
- **Example:** The trainer deployed NGINX using a deployment.yaml that specified 3 replicas, image "nginx:1.16.0", and exposed it on port 80. The matching service.yaml used type LoadBalancer to make it accessible externally.

Container Image Registries

- **Docker Hub vs Azure Container Registry (ACR):** Docker Hub images are typically public and don't require credentials to pull. ACR images are private and require authentication, providing better security for proprietary applications.
- **Analogy:** Similar to GitHub repositories—public repos can be cloned without credentials, but private repos require authentication to access or push changes.

Service Types in Kubernetes

- **ClusterIP:** Internal communication only, not accessible from outside the cluster. Used for backend services that don't need external access.
- **NodePort:** Exposes service on a specific port on each worker node. Suitable for single pod deployments or dev/test scenarios. Maps a container port

directly to a node port.

- **LoadBalancer:** Creates an external load balancer (in cloud environments) that distributes traffic across multiple pod replicas. Best for production deployments with multiple replicas where traffic needs to be balanced.
- **Example:** The trainer explained that with 2+ replicas, LoadBalancer is preferred over NodePort because NodePort targets a single pod while LoadBalancer distributes requests across all available pods.

Introduction to Helm Charts

- **What is Helm:** A package manager for Kubernetes, similar to apt/yum for Linux or npm for Node.js. Helm packages pre-configured Kubernetes resources into reusable "charts" that simplify deployment and management.
- **The deployment challenge:** Without Helm, deploying the same application across dev, QA, staging, and production requires copying and manually modifying YAML files for each environment—a repetitive, error-prone process.
- **Helm's solution:** Provides templates with placeholders that pull values from a separate values.yaml file. One set of templates can be reused across all environments by simply changing the values file.

Helm Chart Components

- **Chart.yaml:** Metadata file containing chart name, description, version (semantic versioning), and application version. This file identifies what the chart deploys and tracks changes.
- **values.yaml:** The configuration file containing all customizable parameters like replica count, image repository, image tag, service type, and ports. This is the primary file that changes between environments.
- **templates/ folder:** Contains Kubernetes manifest files (deployment.yaml, service.yaml, etc.) with Jinja-style templating syntax. Uses Go templating language to reference values from values.yaml.
- **helpers.tpl:** Defines reusable template functions for static information like chart names and labels. Reduces redundancy across template files.

- **charts/ folder:** Contains dependent charts for multi-component applications (e.g., WordPress might depend on MySQL chart).

Helm Templating Syntax

- **.Values reference:** In templates, `{{ .Values.image.repository }}` tells Helm to fetch the value from values.yaml under image → repository.
- **Example:** If values.yaml contains `image: repository: nginx, tag: latest`, the template renders as `image: nginx:latest` in the actual deployment.
- **Helper functions:** Defined in helpers.tpl using `{{- define "nginx-chart.name" -}}` syntax. Called in templates with `{{ include "nginx-chart.name" . }}`.

Helm Releases and Versioning

- **Release:** A deployed instance of a Helm chart. Each installation creates a release with a unique name (e.g., "my-nginx").
- **Revision tracking:** Every helm upgrade creates a new revision number. Helm maintains a history of all changes, enabling easy rollbacks.
- **Example:** First install creates revision 1. Changing replicas from 1 to 2 and upgrading creates revision 2. Each revision is timestamped and stored.

Third-Party Helm Repositories

- **Artifact Hub:** Central repository (artifacthub.io) containing thousands of pre-built Helm charts from vendors like Bitnami, Prometheus, Grafana, and more.
- **Benefits:** No need to write YAML from scratch for common applications. Simply pull charts, customize values.yaml, and deploy.
- **Examples mentioned:** Bitnami charts for NGINX, MySQL, MongoDB, PostgreSQL; Prometheus charts for monitoring; Grafana charts for visualization.

🧪 Practical Exercises or Demonstrations

Creating and Connecting to AKS Cluster

Exercise: Participants followed along as the instructor created an Azure Kubernetes Service cluster with 2 worker nodes, specifying node size (Standard_DS2_v2), enabling autoscaling up to 5 nodes, and selecting Azure CNI networking.

Key learning: Understanding cluster configuration options, subscription limits on VM sizes, and how to connect using `az aks get-credentials` command. Verified connection with `kubectl get nodes`.

Deploying NGINX with YAML Files

Exercise: Created nginxdeploy.yaml from Kubernetes documentation with 3 replicas, nginx:1.16.0 image, and app: nginx labels. Applied with `kubectl apply -f nginxdeploy.yaml`.

Key learning: How deployment specifications translate into running pods. Practiced scaling by modifying replicas in YAML and reapplying, or using `kubectl scale deployment` command.

Creating and Exposing Services

Exercise: Created service.yaml with type LoadBalancer, port 80, and selector matching deployment labels. Applied and verified external IP assignment with `kubectl get svc`.

Key learning: How services find pods through label matching, the difference between port (service port) and targetPort (container port), and how LoadBalancer type gets cloud-assigned external IP.

Accessing Pod Containers

Exercise: Used `kubectl exec -it <pod-name> -- /bin/bash` to enter a running NGINX container. Encountered issues with deprecated image repository, switched to nginx:latest which resolved the problem.

Key learning: Container troubleshooting, understanding containers are lightweight (don't include tools like vim by default), installing packages with apt update/apt install vim, and modifying application files (index.html) to verify changes in browser.

Creating Helm Charts from Scratch

Exercise: Used `helm create nginx-chart` to generate chart skeleton. Modified chart.yaml (metadata) and simplified values.yaml to include only replicaCount, image (repository/tag), service (type/port).

Key learning: Understanding Helm chart directory structure and how `helm create` provides a complete starting template.

Customizing Helm Templates

Exercise: Edited templates/deployment.yaml and templates/service.yaml to use simpler templating syntax. Reviewed how `{{ .Values.replicaCount }}` pulls from values.yaml and `{{ include "nginx-chart.name" . }}` uses helpers.tpl.

Key learning: Separating static configuration (in helpers.tpl) from dynamic values (in values.yaml) for maintainability and reusability.

Installing and Managing Helm Releases

Exercise: Deployed chart with `helm install my-nginx ./nginx-chart`. Verified with `helm list` showing revision 1. Made changes to values.yaml (replica: 2, then image tag: 1.17.0, then chart version: 2.0.0) and upgraded with `helm upgrade my-nginx ./nginx-chart`.

Key learning: Each upgrade increments revision number. `helm history my-nginx` shows complete deployment history with timestamps and status.

Helm Rollback Operations

Exercise: After multiple upgrades reaching revision 5, rolled back to initial deployment with `helm rollback my-nginx 1`. This created revision 6 that matched revision 1's configuration.

Key learning: Rollback creates a new revision (doesn't delete history), making it safe and auditable. Verified rollback restored original replica count and image version.

Helm Linting and Validation

Exercise: Demonstrated `helm lint ./nginx-chart` to validate chart syntax and structure. Intentionally broke indentation to show how linting catches errors before deployment.

Key learning: Always lint charts before deployment, especially in CI/CD pipelines. "0 charts failed" indicates clean chart ready for installation.

Packaging Helm Charts

Exercise: Used `helm package ./nginx-chart` to create nginx-chart-1.0.0.tgz artifact file that can be shared with other teams or pushed to artifact repositories like JFrog.

Key learning: Packaged charts serve as point-in-time snapshots, enabling version control and distribution across teams or organizations.

Using --set Flag for Overrides

Exercise: Upgraded deployment with `helm upgrade my-nginx ./nginx-chart --set replicaCount=3` to override values.yaml without editing the file.

Key learning: Runtime overrides useful for quick testing or environment-specific changes without modifying source files. Common in automated deployments where values are injected by CI/CD tools.

📘 Key Takeaways and Insights

Core Principles

- **Infrastructure as Code:** All Kubernetes configurations should be version-controlled in Git repositories. Manual kubectl edits work but aren't auditable or repeatable.
- **DRY (Don't Repeat Yourself):** Helm charts eliminate the need to copy-paste YAML files across environments. One template, multiple value files.
- **Separation of concerns:** Keep static configurations (templates, helpers) separate from dynamic values (values.yaml). This makes maintenance easier and reduces errors.

Best Practices

- **Always lint before deployment:** Run `helm lint` as part of your CI/CD pipeline to catch syntax errors, indentation issues, and structural problems early.
- **Use semantic versioning:** Chart versions (1.0.0 → 2.0.0) communicate breaking changes. App versions track the actual software being deployed.

- **Label consistency matters:** Service selectors must exactly match deployment labels. A mismatch means services can't find pods, causing failures.
- **Choose the right service type:** Use ClusterIP for internal services, NodePort for dev/testing single pods, and LoadBalancer for production multi-replica deployments.
- **Leverage third-party charts:** Don't reinvent the wheel. Check Artifact Hub for pre-built charts before writing from scratch.
- **Maintain deployment history:** Helm's revision tracking is invaluable for auditing and troubleshooting. Never delete history unless absolutely necessary.

Important Warnings

- **Avoid manual edits in production:** While `kubectl edit deployment` works, it bypasses Helm's version control and creates configuration drift. Always update through Helm.
- **Image tag gotchas:** Using deprecated images can cause repository errors. Always verify image availability and prefer specific tags over "latest" in production.
- **Subscription limits:** Azure subscriptions have VM size restrictions. Check recommended VM sizes for your subscription before creating clusters.
- **Namespace awareness:** Resources deploy to "default" namespace unless specified. In production, use dedicated namespaces for isolation and organization.

Helm vs Traditional Approach

- **Traditional method:** Write/copy YAML files for each environment → manually edit values → apply with kubectl → no version tracking → difficult cleanup (delete each resource manually).
- **Helm method:** Create template once → customize per environment with values files → install/upgrade with Helm → automatic version tracking → single-command cleanup with `helm uninstall`.

Real-World Application Context

- **Banking/regulated industries:** Often restrict direct kubectl access. All deployments happen through Jenkins/CI-CD pipelines that read Helm charts from Git repos.
- **Multi-environment deployments:** Teams maintain separate values files (values-dev.yaml, values-qa.yaml, values-prod.yaml) with environment-specific configurations.
- **GitOps with Argo CD:** When you commit changes to Git (like replica: 5), Argo CD automatically pulls and synchronizes to the cluster—pure pull-based deployment model.

✅ Action Items / Next Steps

Practice Activities

- **Create your own AKS cluster:** Follow the Azure portal steps demonstrated in class. Practice creating clusters with different node configurations and networking options.
- **Recreate the NGINX deployment:** Use the YAML files shared in chat to deploy NGINX from scratch. Practice scaling, updating images, and exposing services.
- **Build a custom Helm chart:** Create a Helm chart for a different application (Apache, Redis, or your own containerized app). Include deployment and service templates.
- **Experiment with values files:** Create separate values-dev.yaml and values-prod.yaml with different replica counts and image tags. Practice installing/upgrading with `-f` flag.
- **Practice rollback scenarios:** Install a chart, make several upgrades, then practice rolling back to different revisions. Observe what happens to pod configurations.

Files and Resources to Review

- **YAML files from session:** Review nginxdeploy.yaml, service.yaml, and persistent volume examples shared in chat during class.
- **Kubernetes documentation:** Study the official deployment and service documentation links provided. Understand all available fields and options.
- **Helm documentation:** The instructor will share two documentation files—one on creating Helm charts from scratch, another on using third-party charts.
- **Artifact Hub:** Explore artifacthub.io to browse available charts. Look at Bitnami repository for commonly used applications.

Upcoming Session Preparation

- **Install Helm locally:** Follow the Helm installation instructions for your OS (provided in Helm-starter.txt file shared in chat).
- **Prepare for Prometheus/Grafana:** Next session will cover deploying monitoring stack using third-party Helm charts and building custom dashboards.
- **Docker Desktop/Minikube:** Ensure you have a local Kubernetes environment ready (Docker Desktop with Kubernetes enabled or Minikube) for practicing Helm commands.

Key Concepts to Master

- **Port mapping:** Understand the relationship between container port, targetPort, port, and nodePort in service definitions.
- **Label selectors:** Practice creating matching labels between deployments and services. Troubleshoot scenarios where they don't match.
- **Helm templating:** Study the syntax for `{{ .Values.* }}`, `{{ include }}`, and conditional logic in templates.
- **Kubectl commands:** Master `kubectl get/describe/logs/exec` for troubleshooting deployed applications.

Reflection Questions

- **How would Helm charts simplify your current deployment processes?** Think about projects or environments you work with.

- **What configuration values would differ between your dev and prod environments?** Consider replicas, resources, images, ingress rules.
- **When would you choose third-party charts vs creating your own?** Evaluate the trade-offs of customization vs convenience.
- **How could you integrate Helm into a CI/CD pipeline?** Consider where linting, packaging, and upgrade commands would fit.

Extended Learning

- **Explore Helm hooks:** Research pre-install, post-install, pre-upgrade hooks for running jobs during deployment lifecycle.
- **Study chart dependencies:** Learn how to declare and manage dependent charts in Chart.yaml.
- **Investigate Helmfile:** Tool for managing multiple Helm releases declaratively across environments.
- **Kubernetes resource management:** Research requests and limits for CPU/memory to prepare for production deployments.